

ECE 220 Computer Systems & Programming

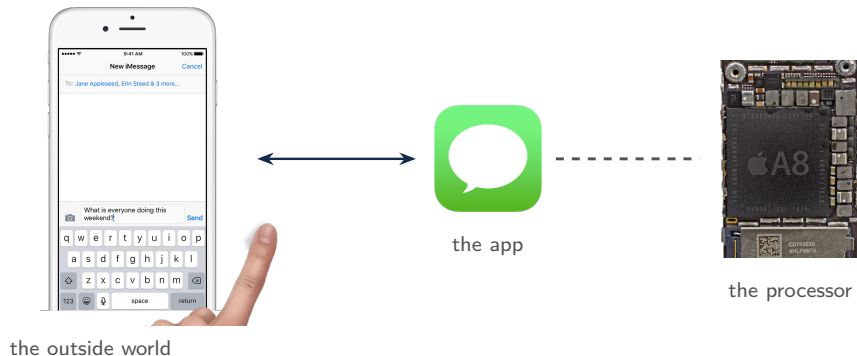
Lecture 1 — Memory-Mapped I/O

January 20, 2026

Prof. Sayan Mitra mitras@illinois.edu Office hours: TBD
Course website: courses.grainger.illinois.edu/ece220/sp2026

The I/O Problem

How do we get data from the outside world into the computer — and back out?



Today's Two Questions

**Your finger is a billion times slower than the processor —
and neither knows when the other is ready.**

1. How does a keystroke actually *reach* the processor?
2. How does the processor *know* a new keystroke has arrived?

By the end of today: both answered, with nothing but loads, stores, and branches. First, a quick LC-3 refresher.

LC-3 Review — ISA (Instruction Set Architecture)

Data type: 16-bit 2's-complement integers

Addressing modes (how the location of an operand is specified):

- Non-memory addresses: immediate (part of the instruction), register
- Memory addresses: PC-relative, base+offset, indirect

Opcodes (16-bit instructions; bits [15:12] specify the opcode):

- *Operate* instructions: `ADD`, `AND`, `NOT`
- *Data movement* instructions: `LD`, `LDI`, `LDR`, `LEA`, `ST`, `STR`, `STI`
- *Control* instructions: `BR`, `JSR/JSRR`, `JMP`, `RET`, `TRAP`, `RTI`
- Condition codes: N (negative), Z (zero), P (positive)

LC-3 Review — Using LD, LDI, LDR, LEA

```
; LC-3 review: LD, LDI, LDR, LEA
.ORIG x3000
    LD    R6, LABEL    ; R6 ← M[LABEL]
    LDI   R6, LABEL    ; R6 ← M[M[LABEL]]
    LDR   R2, R6, #1    ; R2 ← M[R6 + 1]
    LEA   R2, LABEL    ; R2 ← addr of LABEL
LABEL    .FILL x5001
.END
```

Assume the following memory:

Address	Data
x5001	x6001
...	...
x6001	x7001
x6002	x7002

What ends up in the registers?

LD: $R6 \leftarrow M[LABEL] = \text{x5001}$

LDI: $R6 \leftarrow M[M[LABEL]] = \text{x6001}$

LDR: $R2 \leftarrow M[R6+1] = \text{x7002}$

LEA: $R2 \leftarrow \text{address x3004}$

Note: the LDR uses R6 after the LDI ran, so $R6 = \text{x6001}$ and $M[\text{x6002}] = \text{x7002}$.

LC-3 Warm-up

1. Initialize a register to zero

```
AND R0, R0, #0
```

2. Copy a value from R0 to R1

```
ADD R1, R0, #0
```

3. Compute $8 - 2$

```
AND R0, R0, #0    ADD R0, R0, #8    ADD R0, R0, #-2
```

In-Class Problem: Multiplication

Write a program to perform the multiplication 5×4 .

- Need a way to store 5 and 4 as arguments
- There is **no multiply instruction** in LC-3 — so we repeat addition
- Register plan:
 - R0 - result, init to 0
 - R1 - multiplicand, init to 5
 - R2 - loop counter, init to 4

Subtlety: **BRz** reads the condition codes set by the *most recent register write* — here the **ADD R2, R2, #-1** count-down.

```
; Compute 5 * 4 (LC-3 has no multiply!)
.ORIG x3000
; R0 result, R1 multiplicand, R2 counter
    AND R0, R0, #0    ; result = 0
    AND R1, R1, #0
    ADD R1, R1, #5     ; R1 = 5
    AND R2, R2, #0
    ADD R2, R2, #4     ; R2 = 4
LOOP BRz DONE         ; counter zero?
    ADD R0, R0, R1     ; result += R1
    ADD R2, R2, #-1    ; count down
    BR LOOP
DONE HALT
.END
```

Course Logistics

- Lecture: T/R, 11:00 am–12:20 pm CT, ECEB 1002 (except exam days)
- Discussion sections (labs) on Fridays
10 makeup pts/lab worksheet toward MPs (except MP12)
- **MPs:** due Thursdays at 10 pm CT; 100 pts each; late penalty 2 pts/hour; 12 MPs, lowest score (except MP12) dropped
- **Quizzes:** 6 programming quizzes, in person at CBTF; lowest dropped
- **Exams:** 2 midterms + final exam (paper format, in person)
- DRES accommodations and religious observances: follow the instructions on the course website
- Academic integrity: violations lead to FAIR cases

Grading

MPs	15%
CBTF quizzes	20%
Midterm exams	$2 \times 20\%$
Final exam	25%

Textbook: Patt & Patel,
*Introduction to Computing
Systems: From Bits & Gates to
C/C++ & Beyond*, 2nd or 3rd
edition

Tools & Resources

- **Course website:** all relevant course info, MP write-ups, etc.
- **Canvas:** gradebook
- **GitHub:** MP/lab code release and submission (**individual**)
- **Gradescope:** lab worksheets (extra credit) and MP regrades
- **Ed:** discussion board monitored by TAs
- **CBTF** (PrairieTest): quiz reservation tool
- **PrairieLearn:** practice quizzes and CBTF testing platform
- **Resources:** CARE, Counseling Center, DRES

Tips for Academic Success in ECE 220



Keep up with the pace

Every lecture builds on the previous one.



Do the MPs yourself

The quizzes and exams check that you can.



Manage your time well

Start MPs early; 10 pm Thursday comes fast.

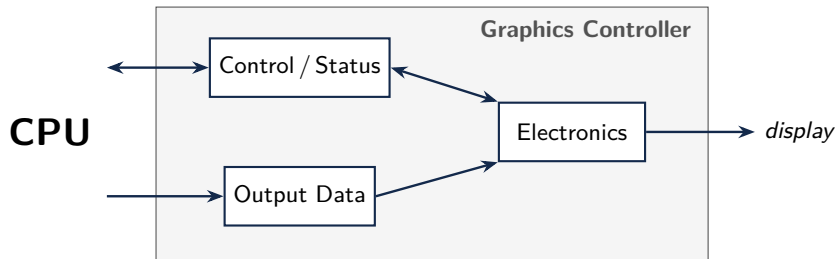
The I/O Problem, Revisited

Back to where we started: the keyboard and the processor live in different worlds, at different speeds.

- The producer of data (finger on a keyboard) works much, *much* more slowly than the consumer of that data (the processor)
- We need to account for ***asynchronous*** operation: the device and the processor run at their own speeds
- We will use a simple **producer/consumer handshake**: the device *signals* when it has (or is ready for) data

Can you think of other I/O devices that you use regularly?

I/O Controller



Control/Status registers

- CPU **tells** the device what to do: **write** to the control register
- CPU **checks** whether the task is done: **read** the status register

Data registers

- CPU transfers data to/from the device

Device electronics

- Performs the actual operation (pixels to screen, character from keyboard)

Memory-Mapped I/O

- Assign a **memory address** to each device register
- Use data movement instructions (load/store) for control and data transfer

LC-3 input and output device registers

KBDR (xFE02) stores the **ASCII value** of the character entered from the **keyboard**

KBSR (xFE00) lets the processor know a new value has been entered

DDR (xFE06) stores the **ASCII value** of the character to be displayed on the **monitor**

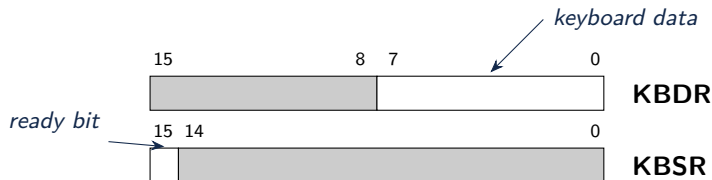
DSR (xFE04) lets the processor know the display is ready for a new value

LC-3 Memory-Mapped Device Registers

Address	Contents	Comments
x0000		; system space
...		
x3000		; user space:
		; programs and data
...		
xFE00	KBSR	; keyboard status
xFE02	KBDR	; keyboard data
xFE04	DSR	; display status
xFE06	DDR	; display data
...		
xFFFF		

- These are the memory **addresses** to which the device registers (KBDR, etc.) are **mapped**
- But the device registers are physically **separate** from the memory
- Memory-mapping device registers is a very common way to design interfaces for computing systems

Read from the Keyboard — Handshaking Using KBDR/KBSR



KBSR[15] controls the synchronization of the slow keyboard and the fast processor.

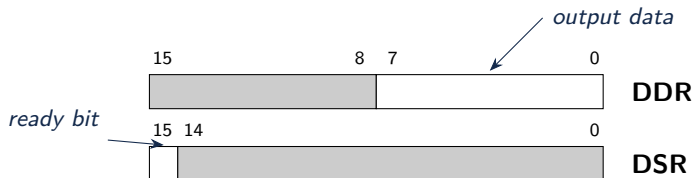
When a key on the keyboard is struck

- ASCII value of the key is placed in KBDR[7:0] (upper bits are zero)
- Ready bit (KBSR[15]) is automatically set to 1 (by the keyboard circuit)
- Keyboard is locked (new character entries will be ignored)

When KBDR is read

- KBSR[15] is automatically set to 0 (by the keyboard circuit)
- Keyboard is unlocked

Output to the Monitor — Handshaking Using DDR/DSR



DSR[15] controls the synchronization of the fast processor and the slow monitor display.

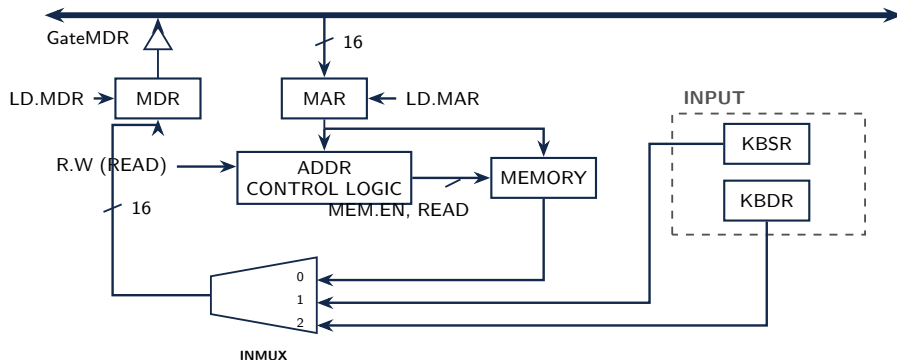
When the monitor is ready to display a new character

- **Ready bit** (DSR[15]) is automatically set to 1 (by the monitor circuits)

When data is written to DDR

- **DSR[15]** is automatically set to 0 (by the monitor circuits), and the character in **DDR** is displayed
- Any other character data written to DDR is **ignored** while DSR[15] is zero

Circuit for Memory-Mapped Input



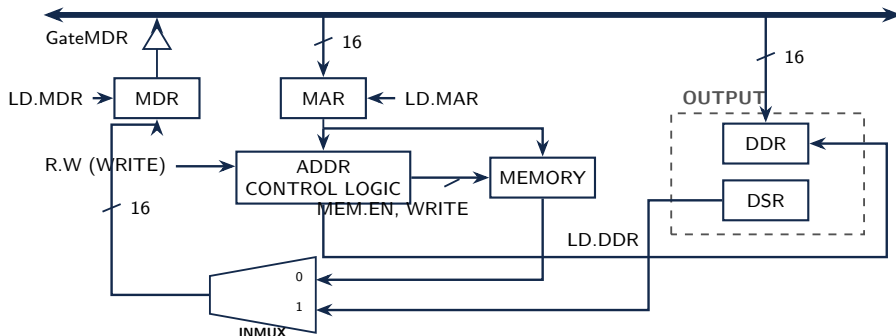
Conventional read LD R0, addr: $MAR \leftarrow \text{addr}$; $MDR \leftarrow \text{Mem}[MAR]$; $R0 \leftarrow MDR$

Memory-mapped input LD R0, xFE02: $MAR \leftarrow \text{xFE02}$; $MDR \leftarrow \text{KBDR}$; $R0 \leftarrow MDR$ — not from memory!

The address decides where the data comes from — xFE02 selects the keyboard, not memory.

Adapted from Y.-C. Hu's SP26 slides; datapath from Patt & Patel.

Circuit for Memory-Mapped Output



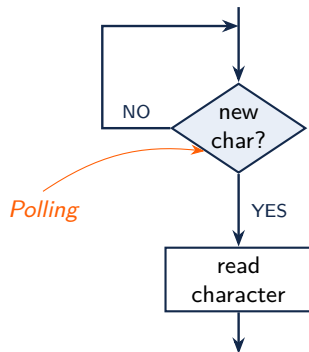
Conventional write `ST R0, addr`: $MAR \leftarrow \text{addr}$; $MDR \leftarrow R0$; $\text{Mem}[MAR] \leftarrow MDR$

Memory-mapped output `ST R0, xFE06`: $MAR \leftarrow \text{xFE06}$; $MDR \leftarrow R0$; DDR $\leftarrow MDR$ — the store never touches memory!

Asynchrony again: when is it safe to store? Poll the ready bit — the code, next.

Adapted from Y.-C. Hu's SP26 slides; datapath from Patt & Patel.

Read from the Keyboard — Basic LC-3 Routine



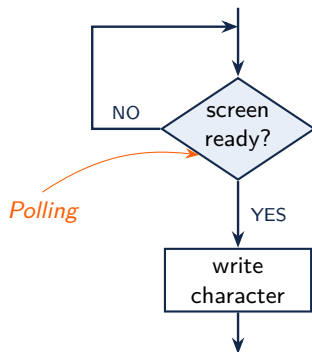
```

; Read one character from the keyboard by polling
; KBSR/KBDR
.ORIG x3000
POLL    LDI      R1, KBSR_ADDR    ; R1 <- KBSR; ready
                                bit is KBSR[15]
        BRzpb    POLL            ; not ready (KBSR
                                [15]=0) -> keep polling
        LDI      R0, KBDR_ADDR    ; R0 <- KBDR: ASCII
                                of the key pressed
        HALT
KBSR_ADDR .FILL    xFE00
KBDR_ADDR .FILL    xFE02
.END

```

`LDI R1, KBSR_ADDR` sets the condition codes: if the ready bit `KBSR[15]` is 1, the value is *negative* — so `BRzpb` loops exactly while there is no new character.

Output to the Monitor — Basic LC-3 Routine



```

; Write one character to the monitor by polling DSR/DDR
.ORIG x3000
    LD      R0, CHAR          ; character to display
POLL    LDI      R1, DSR_ADDR  ; R1 <- DSR; ready bit is
        DSR[15]
        BRzpp POLL           ; display busy (DSR[15]=0)
        -> keep polling
        STI      R0, DDR_ADDR  ; DDR <- R0: character
        appears on screen
        HALT
CHAR     .FILL    x0041        ; 'A'
DSR_ADDR .FILL    xFE04
DDR_ADDR .FILL    xFE06
.END
  
```

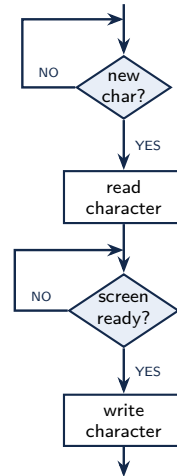
Same polling pattern as the keyboard — but here we are the producer and the display is the slow consumer.

Echo Routine Implementation

```

; Echo: read one character from the keyboard, print it to
; the monitor
.ORIG x3000
INPOLL  LDI      R1, KBSR_ADDR    ; wait for a new
; character
        BRzp    INPOLL
        LDI      R0, KBDR_ADDR    ; R0 ← ASCII of the key
; pressed
OUTPOLL LDI      R1, DSR_ADDR      ; wait for the display
; to be ready
        BRzp    OUTPOLL
        STI      R0, DDR_ADDR      ; echo the character
        HALT                    ; (loop back to INPOLL
; to echo forever)
KBSR_ADDR .FILL  xFE00
KBDR_ADDR .FILL  xFE02
DSR_ADDR  .FILL  xFE04
DDR_ADDR  .FILL  xFE06
.END

```



Summary & What's Next

Today

- **The I/O problem:** devices and the CPU run at different speeds, asynchronously — and neither knows when the other is ready
- **Conceptual solution:** memory-mapped device registers (KBSR xFE00, KBDR xFE02, DSR xFE04, DDR xFE06) plus a ready-bit **handshake**
- **Detailed solution:** the address control logic steers loads/stores to the device registers; **polling** loops busy-wait on the ready bit — keyboard input, monitor output, echo

Next lecture

- TRAPs: asking the operating system to do I/O for us

Code from today: `mult.asm`, `kb_poll.asm`, `out_poll.asm`, `echo.asm` in the course code repository.